CÓDIGOCLARO · RUTA JAVASCRIPT

Tema 10 · JSON y APIs

Guía de estudio · DAM/DAW

JSON.parse/stringify, fetch, códigos HTTP y cabeceras.

1. Explicación

JSON.parse convierte texto en un valor JS; JSON.stringify serializa valores compatibles. fetch devuelve una Promise de Response y no rechaza solo por recibir 404/500: comprueba response.ok. Revisa tipo de contenido y valida los datos antes de usarlos.

2. Ejemplo

Código · javascript

```
const texto = '{"nombre":"Luz","activo":true}';
const dato = JSON.parse(texto);
console.log(dato.nombre, JSON.stringify(dato));
```

Resultado esperado

```
Luz {"nombre":"Luz","activo":true}
```

3. Errores frecuentes

- No manejar SyntaxError de JSON inválido.
- Suponer que fetch rechaza en un estado HTTP 404.
- Esperar que JSON conserve funciones o undefined.

4. Ejercicio

Parsea un registro, acepta solo si activo es true y devuelve JSON serializado.

Casos que debe superar

Entrada	Resultado esperado
{"activo":true}	se acepta
{"activo":false}	se rechaza
texto JSON incompleto	SyntaxError controlado

5. Solución explicada

Código · javascript

```
function leerRegistro(texto) {
  const dato = JSON.parse(texto);
  if (dato.activo !== true) throw new Error("Registro inactivo");
  return JSON.stringify(dato);
}
console.log(leerRegistro('{"activo":true}'));
```

Otras soluciones válidas

- Usar un esquema de validación para objetos con muchas reglas.
- Separar validación de datos y petición HTTP.

6. Fuentes oficiales

MDN — Fetch API · MDN — JSON